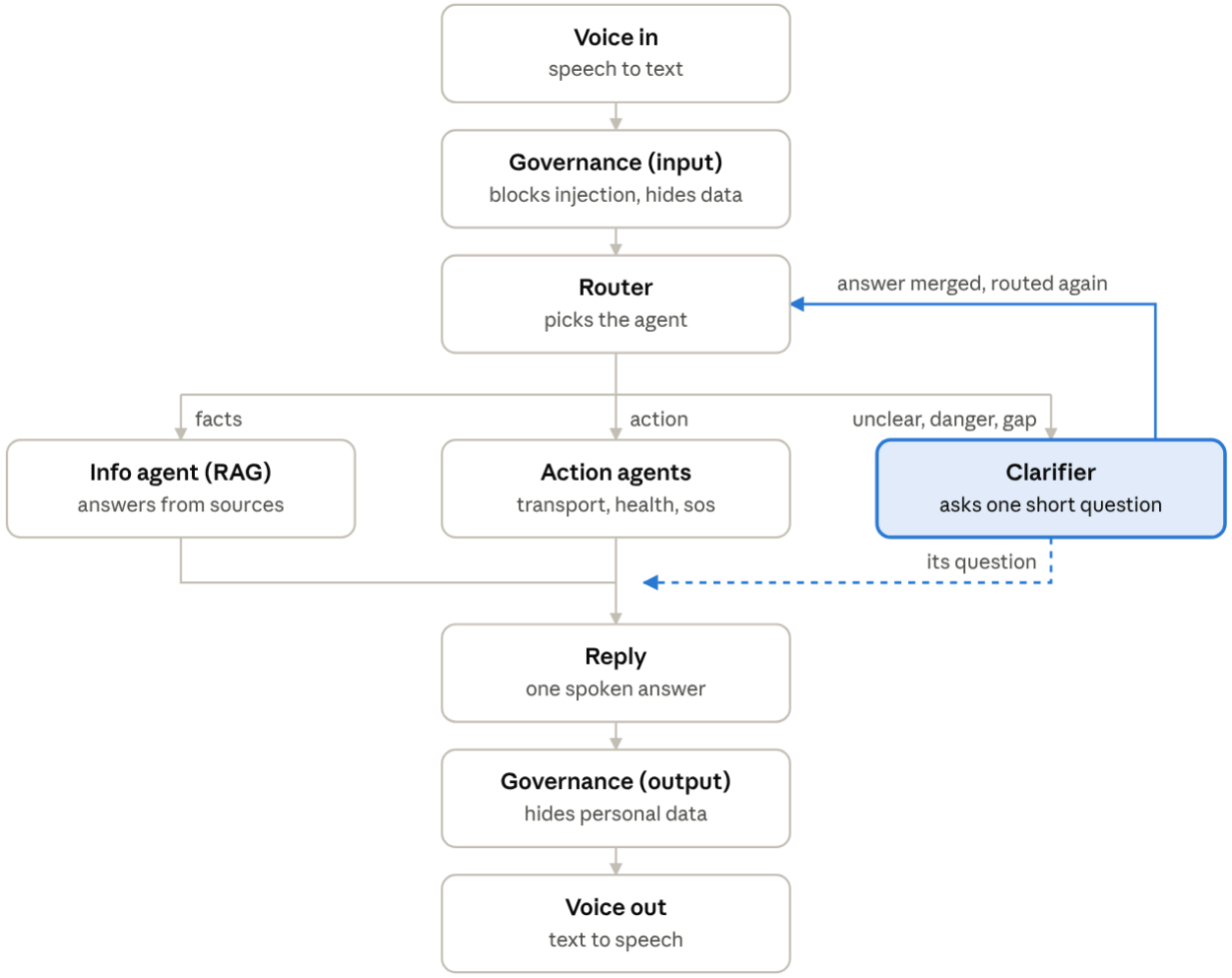
The Clarifier Agent — Team Guide

Nov 10, 2026 · @Anjali

The Clarifier is the JARVIS agent that steps in when a request is unclear, missing a detail, or sounds like danger. It asks a short question, adds the answer to the request, and **keeps asking until the request is complete**. Danger words, "never mind" or a new question end the loop at any time. It is **one standalone file**, `clarifier.py`, which works in any project on its own. The code is in **jarvis_code_with_clarifier.zip**, shared with this guide.

Where it sits

The Clarifier is the safety net under the Router



Blue: the Clarifier. Its question is spoken; your answer goes back through the Router.

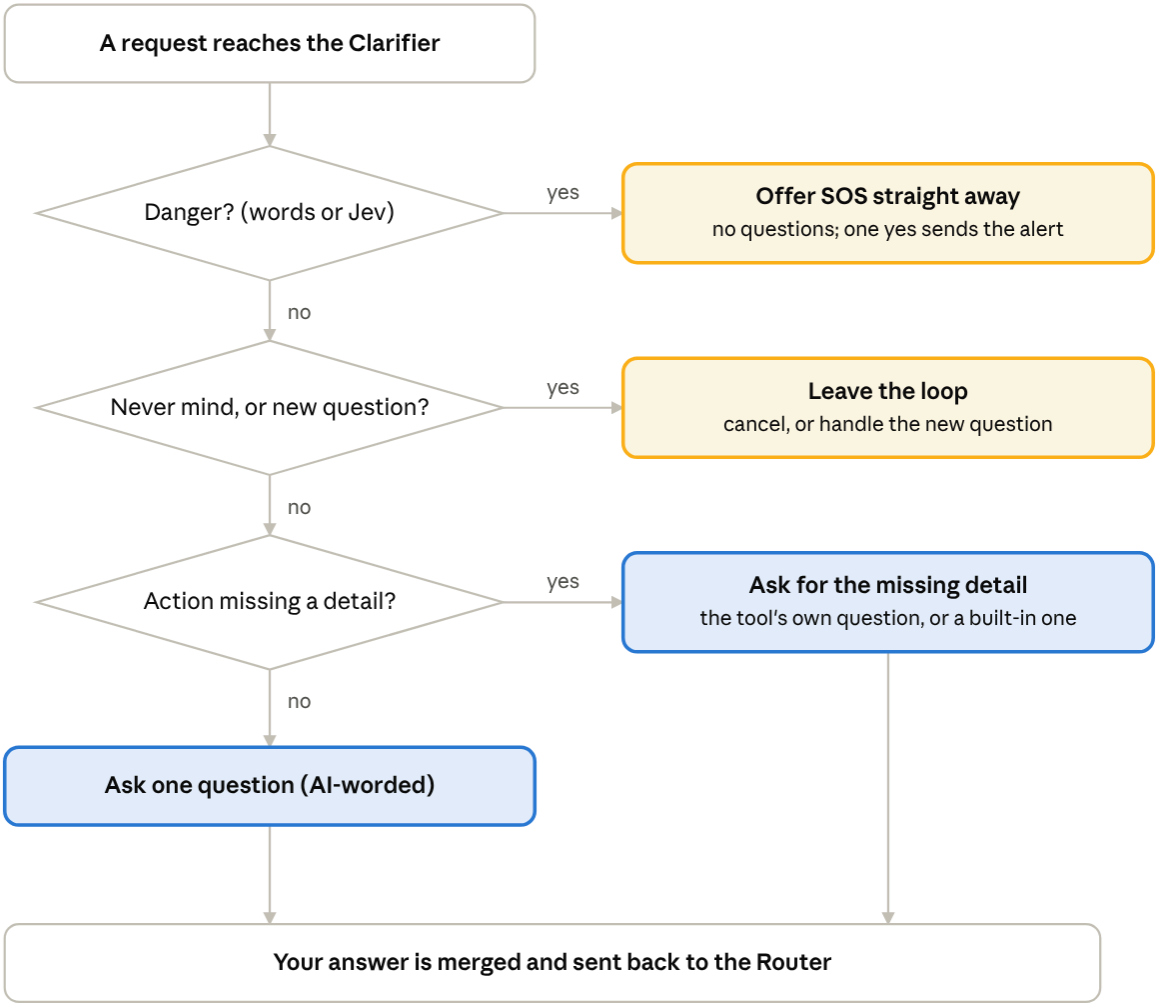
JARVIS pipeline · where the Clarifier sits

Every spoken request goes through the Router. If the request is clear, it goes to an agent. If it is unclear, sounds like danger, or is missing a detail, the Clarifier takes it, asks one question, and sends your answer back to the Router. It is also the last resort: if every other

agent gives up, the Clarifier asks instead of failing silently.

How it decides

Danger first, then the ways out, then what is missing



Orange: help instead of a question. Blue: one short question.

Clarifier decision flow · 3 checks, in this order

The order matters: safety first, then the ways out, then the question. Someone in danger is never asked anything. When an AI model is available it writes each question to fit the conversation; the built-in question is the backup.

Example conversations

These are the built-in questions. With an AI model available, the wording changes to fit the conversation, for example “Where are you heading, and roughly when will you be home?”. Every AI question is checked first: one short question, nothing about card

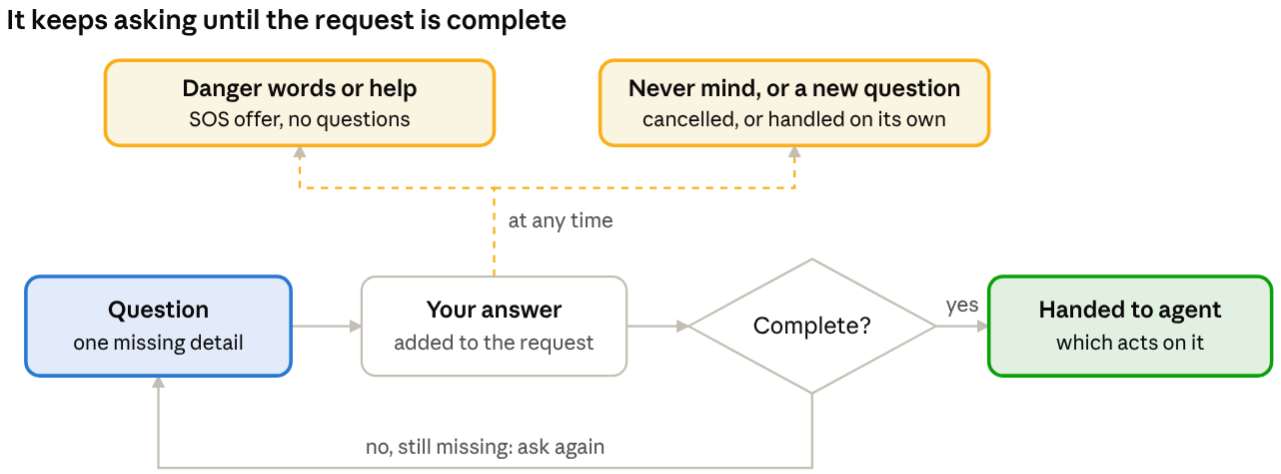
numbers, PINs or codes. If it fails the check or the AI is down, the built-in question below is used. The danger message and the scam warning are never reworded.

You say	What the Clarifier does	JARVIS says
“I'm going out”	Check-in, missing place and time	“Where are you going, and when will you be back?”
“in ten minutes”	Time is now filled; only place is missing	“Where are you going?”
“Le Pouce”	Both filled: sends “I'm going out. In ten minutes. Le Pouce” back to the router	(the check-in agent takes over)
“I'm heading to Le Pouce”	Place given, time missing	“When should I expect you back?”
“Tell my family”	Message missing	“What would you like me to tell them?”
“Is this a scam?”	The message to check is missing	“Please read me the message you received. Don't read out any card numbers or codes.”
“I'll be late”	How long is missing	“By how long?”
“someone is following me”	Danger: no questions	“It sounds like you may be in danger. Should I alert your emergency contacts now? If you can, call 999.”
“never mind” (after a question)	Cancels	“Okay, cancelled.”
“What should I do in a cyclone class 3?” (after a question)	A new question, so it drops the old one	(answered by the info agent)

When it stays out of the way: a complete request such as “Leaving Ebene, home by eleven” or “When is the next bus?” goes straight to the right agent. The Clarifier is never involved.

Any tool, any theme: “Find me a clinic” → the health agent sees its tool needs an area and none was given → Clarifier: “Which area are you in?” → “Rose-Hill” → the health agent finds the clinic. No rule was written for this; the question comes from the tool.

The loop: it keeps asking until it is complete



No fixed number of questions. Every 3rd one adds: If you need urgent help, just say help.

the loop · no fixed number of questions

There is no fixed number of questions. After each answer it checks again, and only hands the request on once nothing is missing. If the same detail is still missing it says “Sorry, I didn’t catch that.” and asks again. Three ways out at any moment: danger words (including just “help”) go to the SOS offer, “never mind” cancels, and a brand-new question is handled on its own. Every 3rd question also adds “If you need urgent help, just say help.”, so someone who is struggling always has a way out. A limit can still be switched on with `max_questions` at the top of `clarifier.py`; it is off by default.

What it asks for

It works for any theme. Every tool already lists the details it needs (find_clinic needs an area). If the user leaves one out, the agent does not guess: it hands over to the Clarifier, which asks for that detail. A tool can carry its own question, for example asks={"area": "Which area are you in?"}; without one, the Clarifier asks “Could you tell me the area?”. So any new tool on the day gets its questions automatically. On top of that, four common actions have fast built-in rules that need no AI at all:

Action	Triggered by	Details it needs	Question if missing
Running late	“I’ll be late”, “extend my trip”	time	“By how long?”
Scam check	“scam”, “is this real?”, “suspicious message”	the message	“Please read me the message you received…”

Action	Triggered by	Details it needs	Question if missing
Tell family	“tell / message / remind my family, mum, son…”	the message	“What would you like me to tell them?”
Check-in	“I'm going out”, “heading to”, “leaving”, “walking home”, “going fishing”, “check on me”	place, time	“Where are you going, and when will you be back?”

Danger phrases (no questions, straight to the SOS offer): “help!”, “someone is following me”, “he is hitting me”, “I'm scared”, “I can't breathe”, “I fell and can't get up”, “call the police”, and in French “au secours”, “à l'aide”, “quelqu'un me suit”, “j'ai peur”, “il me frappe”, “appelez la police”.

They are deliberately specific, so “Can you help me find a clinic?” and “What is the emergency number?” do **not** trigger SOS.

Danger worded in other ways, such as “there's a man behind me and I don't feel ok”, is caught by Jev instead. Jasveer's Jev call now asks one extra yes/no question, “Does the user sound like they are in danger?”, in the same request. At 0.7 or above (`danger_floor` in `config.py`), it goes to the SOS offer.

Adding an action for the theme on the day: write the tool as usual and, optionally, give it a question for each detail: `@tool("find_pharmacy", "...", asks={"area": "Which area are you in?"})`. Nothing else is needed. A built-in `Rule(...)` in `clarifier.py` is only worth adding for a very common action that should be asked without any AI.

Language: English only, as agreed by the team. French support is still in the file but switched off: setting `languages` to `["en", "fr"]` at the top of `clarifier.py` turns it back on. French danger words such as “au secours” are still caught for safety, but the reply is in English.

Run and test it

You need Python 3.10 or newer. No API keys are needed for the tests.

1. Unzip **jarvis_code_with_clarifier.zip** and open a terminal in the `jarvis_code` folder.
2. Run the automatic tests:

```
python test_clarifier.py
```

You should see `23/23 passed`. That tests `clarifier.py` on its own. For the Clarifier inside JARVIS, also run `python test_clarifier_jarvis.py` and expect `21/21 passed`.

3. Talk to it yourself:

```
python agents.py
```

Type at the `You:` prompt, and type `quit` to stop. Restart between tests so each one starts fresh.

4. Optional, for real AI replies: `pip install anthropic`, then set your key (`set ANTHROPIC_API_KEY=...` on Windows, `export ANTHROPIC_API_KEY=...` on Mac) and run step 3 again. Without a key, agent replies show as `[mock:agent-name]`, which is expected.

Try these, one per run:

Type	You should get
`I'm going out` → `in ten minutes` → `dunno` → `no`	Both details asked, then only the place, then the give-up SOS offer, then "Okay, cancelled."
`someone is following me` → `yes`	The danger message, then the SOS agent runs
`I'm going out` → `never mind` → `When is the next bus?`	The question, "Okay, cancelled.", then a normal bus answer
`Can you help me find a clinic in Rose-Hill?`	A normal health answer, with **no** danger message

Important: no real alert is sent. `send_alert` is still a placeholder that only returns "(pretend) alert sent". Even with a real key, Claude may say "I've alerted your contacts", but nobody receives anything until Rushil's WhatsApp code is connected.

Use it in any project

`clarifier.py` is one file with no dependencies: no other project files, no packages, no API keys. Copy it into any Python project and use it in three lines:

```
from clarifier import Clarifier
clarifier = Clarifier()                 # optional:
Clarifier(ask_llm=my_ai_function)
turn = clarifier.step("I'm going out")  # once per user message
```

`turn.action`	What your app does
`ask`	Say `turn.say`, then pass the user's reply to `step()` again
`proceed`	The request is complete: handle `turn.request` (every answer included)
`sos`	Say `turn.say` (the SOS offer) and start your emergency flow
`cancelled`	Say `turn.say`

`ask_llm` is any function that takes a system prompt and a user prompt and returns text, so any AI model works. One of the tests copies only `clarifier.py` into an empty folder and runs it there, to prove it is standalone.

How to tell it is working

The Clarifier writes its own line in the terminal every time it acts, so you can see exactly when it stepped in and why.

Log line	Meaning
`route: danger or missing details -> clarifier`	It caught the request before normal routing
`[clarifier] ask intent=check_in missing=['place', 'time']`	It knows what you want and which details are missing
`[clarifier] ask intent=check_in missing=['place']`	Your answer filled the time; only the place is left
`[clarifier] sos intent=None missing=[]`	Danger words: it skipped the questions and offered SOS
`[clarifier] give_up ...`	It hit the optional question limit (off by default)
`[clarifier] ask intent=None missing=[]`	Unclear and no rule fits, so the AI words a general question
no `[clarifier]` line at all	A clear request went straight to the right agent, which is correct

What changed in the code

Two new files hold the Clarifier. The small changes to Jasveer's files are marked `CLARIFIER` so they are easy to find, review or undo.

File	Change	Why
`clarifier.py` (new)	One standalone file: danger words, rules per action, the loop, AI-written questions, merging answers	All the Clarifier's decisions, testable without any AI
`test_clarifier.py` (new)	23 tests of the file on its own; test_clarifier_jarvis.py has 21 more inside JARVIS	Proves each behaviour works, no keys needed
`agents.py`	`ClarifierAgent` now uses `clarifier.py`; Jasveer's menu question kept as the fallback	Fixed questions for known actions, AI only for vague ones
`agents.py`	New `sos` agent holds `send_alert`; `health` keeps only `find_clinic`	Only alerts ask "Shall I go ahead?", not every clinic search
`orchestrator.py`	`Router.plan`: danger or missing details go to the Clarifier first	So "I'm going out" is never acted on half-finished
`orchestrator.py`	`handle()`: merges your answer into the request and sets up the SOS yes/no	So the next turn has the full request
`orchestrator.py`	`run_chain()`: the Clarifier skips the safety gate	It has no tools; otherwise Jev could block "he is hitting me"
`orchestrator.py`	`Router.keyword()`: no matching words now asks instead of picking the first agent	It was sending unrelated requests to `transport`
`providers.py`	Reads only the text part of Claude's reply	The newer model sends a "thinking" part first, which crashed every Claude call
`config.py`	`danger_floor: 0.7`, `sos_agent: "sos"`	Both settings in one place

Also added: tools take `asks={...}` questions (`agents.py`); an action agent replies `NEED_DETAILS: tool.param` instead of guessing, and `run_chain()` then goes straight to the Clarifier (`orchestrator.py`); one extra Jev question, `is_danger` , in the same call (`orchestrator.py`). The Clarifier's own settings (languages, question limit, help reminder) live at the top of `clarifier.py` , so the file needs nothing else.

What each teammate needs to do next

- [ ] **Jasveer:** review the changes marked `CLARIFIER` in `orchestrator.py` , `agents.py` and `providers.py` , and merge them into the main code.
- [ ] **Rushil:** turn the WhatsApp alerts code into the `send_alert` tool used by the `sos` agent, so alerts are really sent.
- [ ] **Rushil + Jasveer:** add a `checkin` agent for start trip, I'm safe and running late. Until it exists, a complete check-in like “Le Pouce, back by two” has no agent to go to.
- [ ] **Everyone:** after any change, run `python test_clarifier.py` (23/23) and `python test_clarifier_jarvis.py` (21/21).
- [ ] **On the day:** give each new tool an `asks` question. No change to `clarifier.py` is needed.
- [ ] **Open question:** SOS currently needs a “yes” before it sends, because of Jasveer's rule for high-risk tools. Do we keep that, or send immediately for danger phrases?